

claude-windows / 90c0b6b8-7430-4870-98cb-e487eacfa1d7

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\90c0b6b8-7430-4870-98cb-e487eacfa1d7\subagents\workflows\wf_ad5139d3-ec0\agent-a27412ab61a8559b5.jsonl`  
- SHA-256: `28edeca7b67e0ebccb7b0bfa72e144b4d0df7ecab2f9a78aa08c9fd1146dd014`  
- Source modified: `2026-07-04T01:49:23+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_ad5139d3-ec0`  
- session_id: `90c0b6b8-7430-4870-98cb-e487eacfa1d7`
```

Transcript

1. user (2026-07-04T01:48:32.871Z)

Repo root: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. Find the control-plane HTTP route definitions (grep for @app, APIRouter, add_api_route, '/api/' near route decorators; likely under backend/api/ or backend/server*).

Answer precisely, each with file:line evidence:

1. Exact path + method + request body + response body for each of: health check, capabilities, /api/process, job status/poll (/api/jobs/{id}?), /api/compile, download (/api/download/{name}?).
2. /api/process: does it accept {"video_path":"gs://..."}? What exact field name(s) for the input? What id field does the response return for polling?
3. /api/compile: exact request body (is it {video_id, segment_ids}? exact field names) and response.
4. Upload endpoints: exact paths+methods for /upload/init and the part-upload + finalize, and what /api/process expects to receive after an upload completes (a local filesystem path? a token?).

Return structured findings + gaps.

2. assistant (2026-07-04T01:48:34.804Z)

I'll start by finding the control-plane HTTP route definitions.

3. user (2026-07-04T01:48:37.508Z)

Found 4 files

backend\main.py

backend\api\uploads.py

docs\archives\past_projects\AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md

docs\CODE_MAP.md

4. user (2026-07-04T01:48:38.276Z)

Found 48 files

backend\infrastructure\database__jobs.py

backend\main.py

deploy\cloudrun\README.md

tests\test_async_jobs.py

docs\NEXT_STEPS.md

backend\config\startup.py

backend\config\models.py

backend\orchestration.py

tests\test_api_endpoints.py

tests\test_upload_download.py

backend\api\uploads.py

docs\archives\past_projects\AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md

docs\DATA_IN_GCS.md

backend\infrastructure\database.py

docs\CODE_MAP.md
backend\ui\assets\index-Cqr7zJeJ.js
docs\BACKLOG.md
docs\archives\past_projects\KHELSUTRA_GURU_HEALTH_REMEDIATION_PLAN-FOLDED-2026-07-02.md
docs\archives\past_projects\CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md
docs\PLATFORM_ARCHITECTURE.md
tests\test_api_cloud_dispatch.py
backend\api\job_worker.py
CLAUDE.md
tests\test_storage_capabilities.py
backend\storage\capabilities.py
README.md
tests\test_watermark_i18n.py
tests\test_server_entrypoint.py
tests\test_job_worker_loop.py
tests\test_locale_analytics.py
tests\test_config.py
tests\test_edge_auth.py
scripts\verify_compute_path.py
backend\pipeline\stitcher\watermark_i18n.py
docs\archives\past_projects\DECOUPLED_COMPUTE\README.md
docs\archives\past_projects\DECOUPLED_COMPUTE\05-COST-ATTRIBUTION.md
docs\UI_PROPOSAL.md
docs\PACKAGING_PLAN.md
docs\CREATING_REELS.md
docs\CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md
docs\COMPUTE_DECOUPLED_SERVING\README.md
backend\assets\fonts\README.md
backend\api\static\app.js
docs\UI_ALPHA_EXECUTION_PLAN.md
docs\VIDEO_LOCALITY_MODEL.md
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
docs\archives\DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md
tests\test_import_smoke.py

5. assistant (2026-07-04T01:48:41.219Z)

I found the key files. Let me read them.

6. user (2026-07-04T01:48:42.546Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-1240 of 1625 total (27839 tokens, cap 25000). Call Read with offset=1241 limit=1240 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```
1     import argparse
2     import json
3     import logging
4     import os
5     import shutil
6     import sys
7     import tempfile
8     import threading
9     import uuid
10    from importlib.metadata import PackageNotFoundError
11    from importlib.metadata import version as _pkg_version
12    from urllib.parse import quote
13
14    import uvicorn
15    from dotenv import load_dotenv
16
17    # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
    packaged
18    # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
```

```

keeps the
19     # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
20     # stdlib-only import ? keeps this pre-pydantic line light.
21     from backend.utils.app_home import env_file_to_load, resolve_output_dir
22
23     load_dotenv(env_file_to_load())
24
25     # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
26     logging.basicConfig(
27         level=logging.INFO,
28         format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
29         datefmt="%H:%M:%S",
30     )
31     logger = logging.getLogger(__name__)
32     from typing import Any, Dict, List, Optional
33
34     from fastapi import FastAPI, HTTPException, Request
35     from fastapi.middleware.cors import CORSMiddleware
36     from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
37     from fastapi.staticfiles import StaticFiles
38     from pydantic import BaseModel
39     from starlette.middleware.trustedhost import TrustedHostMiddleware
40
41     from backend import (
42         billing, # M8: per-job cost ledger (USD internal / INR display)
43         storage, # M2: URI-aware input acquisition (local = identity passthrough)
44     )
45     from backend.api import (
46         auth as edge_auth, # M9: Cloudflare Access edge-auth (default-OFF)
47     )
48     from backend.config import (
49         AppConfig,
50         StartupCheckError,
51         enforce_startup_checks,
52         write_light_default_config, # P2b: batteries-included first-run config seeding
53     )
54     from backend.config import ( # new typed config
55         load_config as load_app_config,
56     )
57     from backend.infrastructure.database import Database
58     from backend.pipeline.segmenters.base import segmenter_registry
59     from backend.pipeline.stitcher.compiler import VideoCompiler
60     from backend.pipeline.validators.base import registry
61     from backend.providers import provider_registry
62     from backend.reporting import emit_indexer_report
63     from backend.results import Failure # standardized error/result contract
64
65     # Server/network bootstrap helpers (CORS/host-allowlist config + the `indexer --app`
launcher)
66     # live in backend.server_runtime (P24 god-module split). Imported HERE, above `app =
FastAPI(...)` ,
67     # because the module-level add_middleware(...) below calls
_cors_origins_from_env/_allowed_hosts_from_env
68     # at import time; re-exported so `import backend.main as m; m._resolve_bind_host(...)`
keeps working.
69     from backend.server_runtime import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
70         _allowed_hosts_from_env,
71         _cors_origins_from_env,
72         _find_free_port,
73         _open_browser_when_ready,
74         _port_is_free,
75         _resolve_app_port,

```

```

76     _resolve_bind_host,
77 )
78 from backend.storage.capabilities import (
79     storage_capabilities, # P3: secret-free "which media can this box use" report
80 )
81 from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
82     ffmpeg_bin,
83     ffprobe_bin,
84 )
85 from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
86 from backend.utils.hashing import compute_video_id # canonical file-identity hashing
87 from backend.utils.paths import (
88     output_base,
89     resolve_under_root,
90     safe_output_filename,
91     validate_input_path,
92 )
93 from backend.utils.redact import (
94     redact_secrets, # P0b: mask secret-shaped fields in /api/config
95 )
96 from backend.utils.single_instance import acquire_lock # P0e: one instance per database
97
98 app = FastAPI(title="Sports Highlight Indexer API")
99
100
101 def create_app(config: AppConfig, db: Database) -> FastAPI:
102     """Factory: inject shared state via ``app.state`` so route handlers access it
103 through
104     ``request.app.state`` ? eliminating the Optional module-level globals that forced
105     ``backend.main`` and ``backend.api.job_worker`` into mypy quarantine.
106
107     Also sets the module-level ``global_config`` / ``db_instance`` for transitional
108     compatibility (helpers and tests still reference them). Route handlers use
109     ``request.app.state`` exclusively."""
110     global global_config, db_instance
111     global_config = config
112     db_instance = db
113     app.state.config = config
114     app.state.db = db
115     return app
116
117 # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
118 browsers per the
119 # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
120 credentials stay
121 # off. The origin list is configurable via the env var above (default [""]) so a hosted
122 deploy can
123 # lock it to its origin; the middleware is fixed at startup.
124 app.add_middleware(
125     CORSMiddleware,
126     allow_origins=_cors_origins_from_env(),
127     allow_credentials=False,
128     allow_methods=["*"],
129     allow_headers=["*"],
130 )
131
132 # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are
133 honoured
134 # (a remote page can't read responses cross-origin AND its Host header is rejected
135 here). www_redirect
136 # off so an unexpected ``www.`` host is rejected rather than 307-redirection.
137 app.add_middleware(
138     TrustedHostMiddleware,

```

```

135         allowed_hosts=_allowed_hosts_from_env(),
136         www_redirect=False,
137     )
138
139
140     # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
01) ---
141
142
143     # Serves static frontend files directly (now under api/ per approved structure).
144     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an
145     # arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
146     # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
147     _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
148     try:
149         os.makedirs(_STATIC_DIR, exist_ok=True)
150     except OSError:
151         # The dir ships with the package (a no-op create); guard against a read-only install
152         # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
PermissionError.
153         pass
154     app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
155
156     # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
snapshot
157     # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
if it's
158     # absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
no SPA change
159     # is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets
is enough
160     # (no client-side path routing ? no history-fallback catch-all, which would also be a
route-order hazard).
161     _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
162     _UI_DIR = (
163         _BUNDLED_UI_DIR
164         if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
165         else _STATIC_DIR
166     )
167     _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
168     if os.path.isdir(_UI_ASSETS_DIR):
169         app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
170
171
172     @app.get("/", response_class=HTMLResponse)
173     def serve_index():
174         index_path = os.path.join(_UI_DIR, "index.html")
175         if os.path.exists(index_path):
176             with open(index_path, "r", encoding="utf-8") as f:
177                 return f.read()
178         return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"
179
180
181     def _bundled_ui_version() -> Optional[Dict[str, Any]]:
182         """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
bundled.
183         Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
guard)."""
184         path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
185         try:
186             with open(path, "r", encoding="utf-8") as f:
187                 return json.load(f)
188         except (OSError, ValueError):

```

```

189         return None
190
191
192     # Global variables initialized at startup
193     db_instance: Optional[Database] = None
194     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
195     # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
196     # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
197     _instance_lock: Optional[Any] = None
198
199     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
200     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
201     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
202     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
203     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
204     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
205     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
206     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
207         _MAX_DRAIN_WAKE_SEC,
208         JobWaiting,
209         _arm_wake_timer,
210         _drain_due_jobs,
211         _get_executor,
212         _job_to_request,
213         _maybe_record_job_cost,
214         _run_claimed_job,
215         _schedule_next_drain_if_waiting,
216     )
217
218     # The processing/compile ORCHESTRATION (hash?validate?segment?report pipeline, Cloud Run
219     # dispatch+ingest, compile resolution + worker handler) now lives in
backend.orchestration
220     # (P23 god-module split; re-cut of #390). Re-exported here so the names stay addressable
on
221     # backend.main ? `import backend.main as m; m._execute_processing(...)` the job
worker's
222     # call-time `_main._execute_compile(...)` seam, and monkeypatch.setattr(backend.main,
...)
223     # all keep working unchanged.
224     from backend.orchestration import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
225         _cloud_available,
226         _CompileError,
227         _execute_compile,
228         _execute_processing,
229         _finalize_reingest,
230         _ingest_remote_segments,
231         _maybe_dispatch_remote,
232         _remote_input_size_bytes,
233         _report_config_for_input,
234         _resolve_compile,
235     )
236
237
238     # Legacy thin wrapper for any remaining direct calls during transition.
239     # New code should import load_app_config / AppConfig from backend.config directly.
240     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
241         cfg = load_app_config(config_path)
242         return cfg.model_dump(mode="json")
243

```

```

244
245     # --- API Models ---
246     class ProcessRequest(BaseModel):
247         video_path: str
248         player_pool: Optional[List[str]] = None
249         max_frames: Optional[int] = None
250         segmenter: Optional[str] = None
251         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
252         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
253         locale: Optional[str] = None
254         # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
255         # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
256         # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
257         compute: Optional[str] = None
258         # Explicit opt-in reprocess path. By default, a video whose content hash already has
stored
259         # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
260         force: bool = False
261
262
263     class AnnotationsRequest(BaseModel):
264         # Finished rally labels for a video, as the annotator's byte-compatible CSV (P8,
from the Studio
265         # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is
written verbatim.
266         video_id: str
267         csv: str
268
269
270     class AssetRequest(BaseModel):
271         # "Store this video in my medium" (STUDIO_MEDIA_LIFECYCLE.md P4, D14). Exactly ONE
source:
272         #   uploaded_path ? a local file from the chunked upload (the durable copy's source;
kept as the
273         #   local working copy for the run), OR
274         #   source_uri ? a video ALREADY at rest in a medium (a pasted
s3:///gcs:///gdrive:// URI) ?
275         #   registered as-is, no copy (D14b).
276         # medium_uri is the destination store (a bucket/Drive/local folder URI). The asset
is keyed by
277         # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content
hash.
278         uploaded_path: Optional[str] = None
279         source_uri: Optional[str] = None
280         medium_uri: str
281         sport: Optional[str] = None
282
283
284     class QueryRequest(BaseModel):
285         video_id: str
286         server: Optional[str] = None
287         receiver: Optional[str] = None
288         duration_min: Optional[float] = None
289         event_type: Optional[str] = None
290
291
292     class CompileRequest(BaseModel):
293         video_id: str
294         segment_ids: List[int]

```

```

295         output_filename: str
296         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
297         # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
298         locale: Optional[str] = None
299
300
301     # --- API Endpoints ---
302     @app.get("/api/config")
303     def get_config(request: Request):
304         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
305         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
306         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
307         # by tests/test_redact.py + the /api/config redaction test).
308         cfg = request.app.state.config
309         if cfg is None:
310             return {}
311         return redact_secrets(cfg.model_dump(mode="json"))
312
313
314     @app.get("/api/health")
315     def health(request: Request):
316         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
317         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
318         confirm the engine is reachable and which detector is wired. Never raises."""
319         cfg = request.app.state.config
320         sport = getattr(cfg, "sport", None)
321         indexing = getattr(cfg, "indexing", None)
322         return {
323             "status": "ok",
324             "sport": sport,
325             "default_segmenter": getattr(indexing, "default_segmenter", None),
326         }
327
328
329     # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
330     # endpoint's request/response shape changes incompatibly (purely additive endpoints need
no bump).
331     API_VERSION = 1
332
333
334     def _engine_version() -> str:
335         """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""
336         try:
337             return _pkg_version("badminton-highlight-indexer")
338         except PackageNotFoundError:
339             return "unknown"
340
341
342     @app.get("/api/version")
343     def api_version():
344         """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
P1)."""
345         return {
346             "engine_version": _engine_version(),
347             "api_version": API_VERSION,
348             "ui": _bundled_ui_version(),
349         }
350

```



```

351
352     @app.get("/api/capabilities")
353     def capabilities(request: Request):
354         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
355         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
356         value. Best-effort + never raises (the wizard degrades gracefully)."""
357         cfg = request.app.state.config
358         indexing = getattr(cfg, "indexing", None)
359         deployment = getattr(cfg, "deployment", None)
360         ffmpeg = shutil.which(ffmpeg_bin()) # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
361         ffprobe = shutil.which(ffprobe_bin())
362         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
363         wasb = getattr(indexing, "wasb", None)
364         weights_path = (
365             os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
366         )
367         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
368         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
369         # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
370         # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
371         nvidia_smi = shutil.which("nvidia-smi")
372         return {
373             "api_version": API_VERSION,
374             "engine_version": _engine_version(),
375             "segmenters": sorted(segmenter_registry.list_available().keys()),
376             "default_segmenter": getattr(indexing, "default_segmenter", None),
377             "ffmpeg": {
378                 "ffmpeg": bool(ffmpeg),
379                 "ffprobe": bool(ffprobe),
380                 "ffmpeg_path": ffmpeg,
381                 "ffprobe_path": ffprobe,
382             },
383             "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
384             "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
385             "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
386             "deployment": {
387                 "profile": getattr(deployment, "profile", "") or "",
388                 "compute_target": getattr(deployment, "compute_target", "") or "",
389                 # item 3: can this server satisfy a per-request compute="cloud" dispatch?
The SPA shows
390                 # the "run in the cloud" toggle only when true (else it's local-only).
391                 "cloud_available": _cloud_available(cfg),
392             },
393             # P3: which storage media THIS box can store INTO (local always; s3/gcs if
SDK+config;
394             # gdrive if a mount resolves). The P5 medium picker renders only usable
backends. No secrets.
395             "storage": storage_capabilities(
396                 getattr(cfg, "storage", None), local_free_path=output_base(cfg)
397             ),
398         }
399
400
401     def _friendly_video_name(filepath: str) -> str:
402         """A human name for a video from its stored path/URI ? the filename stem
(``~/match_2026.mp4`` ?
403         ``match_2026``). Works for a local path, a bucket key, or a ``gdrive://`` URI
(rsplit on ``/``).
404         Falls back to the whole ``filepath`` if there's no basename."""

```

```

405     base = str(filepath or "").rstrip("/").rsplit("/", 1)[-1].rsplit("\\", 1)[-1]
406     stem = os.path.splitext(base)[0]
407     return stem or base or str(filepath or "")
408
409
410     def _video_dims(validation_results: Any) -> "tuple[Optional[float], Optional[str]]":
411         """Pull `(fps, resolution)` from the persisted validation results, or `(None,
412         None)`. The
413         ``resolution_fps_check`` validator already probed ffprobe and stored
414         ``details={width,height,fps}``
415         on its result (every branch) ? so the videos row carries it with no schema change /
416         re-probe."""
417         for r in validation_results or []:
418             if isinstance(r, dict) and r.get("validator_name") == "resolution_fps_check":
419                 d = r.get("details") or {}
420                 w, h, fps = d.get("width"), d.get("height"), d.get("fps")
421                 resolution = f"{w}x{h}" if w and h else None
422                 return (fps if isinstance(fps, (int, float)) else None), resolution
423         return None, None
424
425     def _video_summary(row: Dict[str, Any]) -> Dict[str, Any]:
426         """Map a DB video row to the API/SPA contract. ADDITIVE ? keeps the raw
427         ``id``/``filepath`` (an
428         existing test + any old client rely on them) and adds the fields the SPA library +
429         the persistent
430         "current video" label actually read: ``video_id`` (the MD5 join key = ``id``), a
431         friendly
432         ``match_name`` (so an uploaded video shows its filename, not a generic fallback),
433         ``local_path``,
434         ``created_at`` (from ``processed_at``), and ``fps``/``resolution`` (from the stored
435         ffprobe
436         validation details) so the library cards can show them. Mirrors the mock engine's
437         shape so mock and
438         real agree."""
439         fp = row.get("filepath") or ""
440         fps, resolution = _video_dims(row.get("validation_results"))
441         return {
442             **row,
443             "video_id": row.get("id"),
444             "match_name": _friendly_video_name(fp),
445             "local_path": fp,
446             "created_at": row.get("processed_at") or row.get("created_at"),
447             "fps": fps,
448             "resolution": resolution,
449         }
450
451     @app.get("/api/videos")
452     def list_videos(request: Request, limit: Optional[int] = None):
453         """List indexed videos (most recent first) so the console survives a reload ?
454         previously only a
455         single-row get existed (PACKAGING_PLAN.md P1). Rows are enriched to the SPA contract
456         (``video_id`` + a friendly ``match_name`` derived from the filename) so the library
457         shows real
458         names and the persistent "current video" label survives an upload ? see
459         ``_video_summary``."""
460         return {
461             "videos": [
462                 _video_summary(v) for v in request.app.state.db.list_videos(limit=limit)
463             ]
464         }
465
466     def _reels_dir(cfg) -> str:

```

```

458         return getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
459
460
461     _REEL_EXTS = (".mp4", ".mov")
462
463
464     @app.get("/api/reels")
465     def list_reels(request: Request):
466         """List compiled highlight reels present in the output dir, newest first, with a
467         download URL.
468         NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ?
469         the engine
470         does NOT track a video_id?reel association today (a per-video listing would be a
471         phantom), so
472         this is a global list. (Per-video reel tracking would need a DB record + compile
473         change ? a
474         later slice.) Only top-level video files are listed (per-video scratch subdirs are
475         skipped)."""
476         base = _reels_dir(request.app.state.config)
477         reels = []
478         try:
479             entries = os.scandir(base)
480         except OSError:
481             return {"reels": []}
482         with entries:
483             for e in entries:
484                 if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
485                     try:
486                         st = e.stat()
487                     except OSError:
488                         continue
489                     reels.append(
490                         {
491                             "filename": e.name,
492                             "size_bytes": st.st_size,
493                             "modified_at": st.st_mtime,
494                             "download_url": f"/api/reels/{quote(e.name)}",
495                         }
496                     )
497             reels.sort(key=lambda r: r["modified_at"], reverse=True)
498         return {"reels": reels}
499
500
501     @app.get("/api/reels/{filename}")
502     def download_reel(filename: str, request: Request):
503         """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
504         output_dir).
505         Mirrors /api/highlights' safety (bare-name + containment checks)."""
506         try:
507             name = safe_output_filename(filename)
508         except ValueError as e:
509             raise HTTPException(status_code=400, detail=str(e)) from e
510         base = _reels_dir(request.app.state.config)
511         path = os.path.join(base, name)
512         try:
513             path = resolve_under_root(path, base)
514         except ValueError as e:
515             raise HTTPException(status_code=400, detail=str(e)) from e
516         if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
517             raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
518         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
519         return FileResponse(path, media_type=media, filename=name)

```

```

517     # --- Annotate-in-UI (P8): serve a video's source + persist finished rally labels
518     -----
519
520     # In-flight annotation-proxy generations ? dedup so a browser's many Range requests to
521     /api/source
522     # don't each kick off a transcode.
523     _proxy_inflight: "set[str]" = set()
524     _proxy_lock = threading.Lock()
525     _source_cache_locks: Dict[str, threading.Lock] = {}
526     _source_cache_locks_guard = threading.Lock()
527
528     def _proxies_dir(cfg) -> str:
529         return os.path.join(output_base(cfg), "proxies")
530
531
532     def _source_cache_dir(cfg) -> str:
533         return os.path.join(output_base(cfg), "source_cache")
534
535
536     def _source_cache_lock(video_id: str) -> threading.Lock:
537         with _source_cache_locks_guard:
538             return _source_cache_locks.setdefault(video_id, threading.Lock())
539
540
541     def _source_cache_path(video_id: str, input_ref, cfg) -> str:
542         ext = os.path.splitext(input_ref.name or "")[1].lower()
543         if not ext or "/" in ext or "\\" in ext or len(ext) > 16:
544             ext = ".mp4"
545         cache_dir = _source_cache_dir(cfg)
546         name = safe_output_filename(f"{video_id}{ext}")
547         return resolve_under_root(os.path.join(cache_dir, name), cache_dir)
548
549
550     def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
551         """Fetch a remote source once into an output-scoped source/proxy cache."""
552         cache_path = _source_cache_path(video_id, input_ref, cfg)
553         if os.path.isfile(cache_path):
554             return cache_path
555         lock = _source_cache_lock(video_id)
556         with lock:
557             if os.path.isfile(cache_path):
558                 return cache_path
559             cache_dir = os.path.dirname(cache_path)
560             os.makedirs(cache_dir, exist_ok=True)
561             need = _remote_input_size_bytes(input_ref, getattr(cfg, "storage", None))
562             require_free_bytes(cache_dir, need, stage="source-cache")
563             tmp = os.path.join(
564                 cache_dir, f".{os.path.basename(cache_path)}.{uuid.uuid4().hex}.tmp"
565             )
566             try:
567                 storage.fetch(input_ref.uri, tmp, config=_storage_settings(cfg))
568                 os.replace(tmp, cache_path)
569             finally:
570                 try:
571                     if os.path.exists(tmp):
572                         os.remove(tmp)
573                 except OSError:
574                     pass
575             return cache_path
576
577
578     def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
579         """Background, best-effort: generate the annotation proxy ONCE so subsequent

```

```

/api/source loads
580     serve the lighter, seekable copy. Deduped; never blocks the request; the source
keeps serving if
581     it fails."""
582     with _proxy_lock:
583         if video_id in _proxy_inflight:
584             return
585         _proxy_inflight.add(video_id)
586     try:
587         from backend.utils.annotation_proxy import make_annotation_proxy, needs_proxy
588
589         if not needs_proxy(src_local):
590             return # already light enough ? no proxy worth making
591         make_annotation_proxy(
592             src_local, os.path.join(_proxies_dir(cfg), f"{video_id}.mp4")
593         )
594     except Exception as e: # noqa: BLE001 - warming is best-effort
595         logger.warning("proxy: warm failed for %s: %s", video_id, e)
596     finally:
597         with _proxy_lock:
598             _proxy_inflight.discard(video_id)
599
600
601     @app.get("/api/source/{video_id}")
602     def get_source(video_id: str, request: Request):
603         """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub.
Serves a
604         frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels
still keyed
605         to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy
in the
606         background for next time ? non-blocking, so the first load is never delayed. Remote
sources are
607         fetched once into an output-scoped source cache, then served and proxied from that
stable copy."""
608         rec = request.app.state.db.get_video(video_id)
609         if not rec or not rec.get("filepath"):
610             raise HTTPException(status_code=404, detail="Video not found.")
611         cfg = request.app.state.config
612
613         # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
614         try:
615             proxy_name = safe_output_filename(f"{video_id}.mp4")
616             proxy_path = resolve_under_root(
617                 os.path.join(_proxies_dir(cfg), proxy_name), _proxies_dir(cfg)
618             )
619         except ValueError:
620             proxy_path = ""
621         if proxy_path and os.path.isfile(proxy_path):
622             return FileResponse(proxy_path, media_type="video/mp4")
623
624         # 2) Otherwise resolve + serve the source.
625         try:
626             input_ref = storage.resolve_input_ref(
627                 rec["filepath"], getattr(cfg, "storage", None)
628             )
629         except (
630             ValueError
631         ) as e: # unsupported scheme etc. ? a clean client error, never a 500
632             raise HTTPException(status_code=400, detail=str(e)) from e
633         except Exception as e: # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced
(not swallowed)
634             raise HTTPException(
635                 status_code=502, detail=f"Could not resolve the source video: {e}"
636             ) from e

```

```

637
638     if input_ref.backend == "local":
639         local = input_ref.key
640     else:
641         try:
642             local = _cached_remote_source(video_id, input_ref, cfg)
643         except HTTPException:
644             raise
645     except Exception as e: # noqa: BLE001 - a fetch miss is surfaced (not
swallowed)
646         raise HTTPException(
647             status_code=502, detail=f"Could not resolve the source video: {e}"
648         ) from e
649     if not os.path.isfile(local):
650         raise HTTPException(status_code=404, detail="Source video file not found.")
651
652     # 3) Warm a proxy in the background for next time.
653     threading.Thread(
654         target=_warm_annotation_proxy, args=(video_id, local, cfg), daemon=True
655     ).start()
656
657     media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
658     return FileResponse(local, media_type=media)
659
660
661 @app.post("/api/annotations")
662 def post_annotations(req: AnnotationsRequest, request: Request):
663     """Persist finished rally labels as the annotator CSV (P8), written verbatim to
664     ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5
665     ``video_id`` ? so it joins
666     the video by content hash exactly as the training pipeline expects. Byte-compatible
with the VLC
667     plugin + browser extension. A later step (owner-gated) promotes labels into the
durable corpus.
668
669     Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is
validated as a bare
670     filename so an untrusted value can't escape the labels dir (arbitrary write)."""
671     try:
672         edge_auth.resolve_tenant(request.headers, request.app.state.config)
673     except edge_auth.EdgeAuthError as e:
674         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
675
676     cfg = request.app.state.config
677     labels_dir = os.path.join(output_base(cfg), "labels")
678     try:
679         name = safe_output_filename(f"{req.video_id}.rallies.csv")
680     except ValueError as e:
681         raise HTTPException(status_code=400, detail=f"invalid video_id: {e}") from e
682     os.makedirs(labels_dir, exist_ok=True)
683     path = os.path.join(labels_dir, name)
684     try:
685         path = resolve_under_root(path, labels_dir)
686     except ValueError as e:
687         raise HTTPException(status_code=400, detail=str(e)) from e
688     with open(path, "w", encoding="utf-8", newline="") as f:
689         f.write(req.csv or "")
690     # Count CSV data rows (a line starting with an integer rally_number) for an honest
ack.
691     num_rows = sum(
692         1 for ln in (req.csv or "").splitlines() if ln[:1].isdigit() and "," in ln
693     )
694     logger.info(
695         "annotations saved: video_id=%s rows=%d -> %s", req.video_id, num_rows, path
696     )

```

```

696         return {"status": "ok", "num_rows": num_rows, "csv_uri": f"labels/{name}"}
697
698
699     def _storage_settings(cfg) -> Dict[str, Any]:
700         """The storage config as a plain settings dict (NO secrets ? see StorageConfig)."""
701         sc = getattr(cfg, "storage", None)
702         if sc is None:
703             return {}
704         return sc.model_dump() if hasattr(sc, "model_dump") else dict(sc)
705
706
707     def _asset_dest_uri(medium_uri: str, video_id: str, ext: str) -> str:
708         """The durable, MD5-keyed destination inside the chosen medium:
709         ``<medium>/<video_id><ext>``. """
710         return f"{medium_uri.rstrip('/')}/{video_id}{ext}"
711
712
713     def _medium_local_dir(medium_ref, storage_settings: Dict[str, Any]) -> Optional[str]:
714         """The LOCAL directory a ``put`` into this medium would write into (so a free-space
715         BLOCK can be enforced, D11), or ``None`` for a cloud bucket (which is cost/quota-warned, never
716         blocked).
717         ``local`` ? the folder; ``gdrive`` ? the resolved path under the Drive mount. """
718         if medium_ref.backend == "local":
719             return medium_ref.key or "."
720         if medium_ref.backend == "gdrive":
721             from backend.storage.capabilities import _gdrive_mount_root
722
723             root = _gdrive_mount_root(storage_settings)
724             if not root:
725                 return None
726             rel = medium_ref.key.lstrip("/")
727             return os.path.join(root, *rel.split("/")) if rel else root
728         return None # s3 / gcs ? a bucket has no readable local capacity
729
730
731     @app.post("/api/assets", status_code=201)
732     def create_asset(req: AssetRequest, request: Request):
733         """Store a video into the user's chosen medium and register it as a library asset
734         (P4, D14).
735
736         SELF-HOST posture only (D7): refused with 403 when ``security.allowed_video_root``
737         is set ? the
738         hosted alpha jails inputs and has no per-user medium credentials, so it stays
739         upload-to-local.
740         Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a
741         local file from
742         the chunked upload) OR a ``source_uri`` (a video already at rest in a medium). An
743         uploaded file +
744         a REMOTE medium ? a durable, MD5-keyed copy is ``put`` into the medium while the
745         local upload
746         stays as the working copy; a ``local`` medium keeps the upload in place (it's
747         already on this box);
748         a pasted at-rest URI is registered as-is (no copy, D14b). The asset is keyed by the
749         whole-file
750         MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears
751         in
752         ``GET /api/videos``. """
753         try:
754             owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
755         except edge_auth.EdgeAuthError as e:
756             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
757
758         cfg = request.app.state.config
759         sec = getattr(cfg, "security", None)

```

```

749     if getattr(sec, "allowed_video_root", None):
750         raise HTTPException(
751             status_code=403,
752             detail=(
753                 "storing into a custom medium is disabled on this hosted deployment "
754                 "(security.allowed_video_root is set); upload to local instead."
755             ),
756         )
757     if bool(req.uploaded_path) == bool(req.source_uri):
758         raise HTTPException(
759             status_code=400,
760             detail="provide exactly one of 'uploaded_path' or 'source_uri'.",
761         )
762     if req.uploaded_path and not req.medium_uri:
763         raise HTTPException(
764             status_code=400, detail="'medium_uri' is required when uploading a file."
765         )
766
767     storage_settings = _storage_settings(cfg)
768
769     # 1) Resolve a LOCAL working copy of the source + its whole-file MD5 (the join key,
770     # D3).
771     if req.uploaded_path:
772         try:
773             validate_input_path(req.uploaded_path, allowed_root=None)
774             src_ref = storage.resolve_input_ref(req.uploaded_path, storage_settings)
775         except ValueError as e:
776             raise HTTPException(status_code=400, detail=str(e)) from e
777         if src_ref.backend != "local" or not os.path.isfile(src_ref.key):
778             raise HTTPException(
779                 status_code=404,
780                 detail=f"uploaded file not found at '{req.uploaded_path}'",
781             )
782         local_working = src_ref.key
783     else:
784         try:
785             validate_input_path(req.source_uri, allowed_root=None)
786             src_ref = storage.resolve_input_ref(req.source_uri, storage_settings)
787         except ValueError as e:
788             raise HTTPException(status_code=400, detail=str(e)) from e
789         # A remote at-rest source is fetched to scratch to hash it (D3) ? reserve that
790         # scratch (P2).
791         if src_ref.backend != "local":
792             need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage", None))
793             require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
794         try:
795             local_working = storage.acquire_local_input(
796                 req.source_uri, getattr(cfg, "storage", None)
797             )
798         except ValueError as e:
799             raise HTTPException(status_code=400, detail=str(e)) from e
800         except Exception as e: # noqa: BLE001 ? a fetch miss is a 502, surfaced not
801             # swallowed
802             raise HTTPException(
803                 status_code=502, detail=f"could not resolve source_uri: {e}"
804             ) from e
805         if not os.path.isfile(local_working):
806             raise HTTPException(
807                 status_code=404, detail="source_uri did not resolve to a readable file."
808             )
809
810     cleanup_raw = req.source_uri or ""
811     try:
812         video_id = compute_video_id(local_working)
813         ext = os.path.splitext(local_working)[1] or ".mp4"

```



```

811
812     # 2) Decide the durable location + copy only when needed.
813     copied = False
814     if req.source_uri:
815         stored_uri = (
816             req.source_uri
817             ) # already at rest in its medium ? no copy (D14b)
818         medium_backend = src_ref.backend
819     else:
820         try:
821             medium_ref = storage.parse_uri(req.medium_uri)
822         except ValueError as e:
823             raise HTTPException(
824                 status_code=400, detail=f"invalid medium_uri: {e}"
825             ) from e
826         medium_backend = medium_ref.backend
827         if medium_ref.backend == "local":
828             # "This computer" ? the upload already lives here; it IS the durable
829             copy (no duplicate).
830             stored_uri = os.path.abspath(local_working)
831         else:
832             dest = _asset_dest_uri(req.medium_uri, video_id, ext)
833             # Free-space: BLOCK a local/Drive-mount destination (D11); a cloud
834             bucket is warn-only.
835             local_dir = _medium_local_dir(medium_ref, storage_settings)
836             if local_dir is not None:
837                 require_free_bytes(
838                     local_dir, os.path.getsize(local_working), stage="store"
839                 )
840             else:
841                 logger.info(
842                     "[ASSETS] cloud medium %s ? capacity not blocked (cost/quota
843                     warn only).",
844                     medium_ref.backend,
845                 )
846             try:
847                 stored_uri = storage.put(
848                     local_working, dest, config=storage_settings
849                 )
850                 copied = True
851             except ValueError as e:
852                 raise HTTPException(status_code=400, detail=str(e)) from e
853             except Exception as e: # noqa: BLE001 ? a put failure is surfaced,
854                 never silently dropped
855                 logger.error(
856                     "[ASSETS] put failed video_id=%s dest=%s: %s",
857                     video_id,
858                     dest,
859                     e,
860                     exc_info=True,
861                 )
862                 raise HTTPException(
863                     status_code=502, detail=f"could not store into the medium: {e}"
864                 ) from e
865
866     # 3) Register the asset (keyed by MD5) so it appears in GET /api/videos ("my
867     videos").
868     if not request.app.state.db.add_video(video_id, stored_uri, "stored", []):
869         raise HTTPException(status_code=500, detail="failed to register the asset.")
870     logger.info(
871         "[ASSETS] stored video_id=%s medium=%s uri=%s copied=%s owner=%s",
872         video_id,
873         medium_backend,
874         stored_uri,
875         copied,

```

```

871         owner_id,
872     )
873     return JsonResponse(
874         status_code=201,
875         content={
876             "video_id": video_id,
877             "medium": medium_backend,
878             "stored_uri": stored_uri,
879             "copied": copied,
880             "sport": req.sport,
881             "status": "stored",
882         },
883     )
884     finally:
885         if cleanup_raw:
886             storage.cleanup_acquired_local_input(
887                 cleanup_raw, local_working, getattr(cfg, "storage", None)
888             )
889
890
891 @app.post("/api/process", status_code=202)
892 def process_video(req: ProcessRequest, request: Request):
893     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
894
895     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
896     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
897     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
898     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
899     """
900     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
901     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
902     # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
903     try:
904         owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
905     except edge_auth.EdgeAuthError as e:
906         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
907
908     allowed_root = getattr(
909         getattr(request.app.state.config, "security", None), "allowed_video_root", None
910     )
911     try:
912         validate_input_path(req.video_path, allowed_root=allowed_root)
913         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
914         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
915         input_ref = storage.resolve_input_ref(
916             req.video_path, getattr(request.app.state.config, "storage", None)
917         )
918     except ValueError as e:
919         raise HTTPException(status_code=400, detail=str(e)) from e
920     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
921     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
922     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
923     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
924     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
925     # rejects a non-local URI as out-of-root.
926     if input_ref.backend == "local" and not os.path.exists(input_ref.key):

```

```

927         raise HTTPException(
928             status_code=404, detail=f"Video file not found at '{req.video_path}'"
929         )
930     # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
931     # if this box can't hold that copy. Best-effort ? an unstatable remote
(network/SDK) or an
932     # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
933     # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
934     if input_ref.backend != "local":
935         need = _remote_input_size_bytes(
936             input_ref, getattr(request.app.state.config, "storage", None)
937         )
938         require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
939     if req.segmenter and not segmenter_registry.get(req.segmenter):
940         available = list(segmenter_registry.list_available().keys())
941         raise HTTPException(
942             status_code=400,
943             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
944         )
945
946     job_id = uuid.uuid4().hex
947     if not request.app.state.db.create_job(
948         job_id,
949         req.video_path,
950         req.segmenter,
951         req.player_pool,
952         req.max_frames,
953         owner_id=owner_id,
954         locale=req.locale,
955         compute=req.compute,
956         force=req.force,
957     ):
958         raise HTTPException(status_code=500, detail="Failed to persist job record.")
959     _get_executor().submit(
960         _drain_due_jobs, request.app.state.config, request.app.state.db
961     )
962     return JSONResponse(
963         status_code=202,
964         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
965     )
966
967
968     @app.get("/api/jobs/{job_id}/cost")
969     def get_job_cost(job_id: str, request: Request):
970         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
971         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
972         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
973         cost = request.app.state.db.get_job_cost(job_id)
974         if cost is None:
975             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
976         fx = request.app.state.config.billing.fx_inr_per_usd
977         usd = cost.get("cost_usd")
978         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
979         cost["fx_inr_per_usd"] = fx
980         return cost
981
982
983     @app.get("/api/videos/{video_id}/cost")
984     def get_video_cost(video_id: str, request: Request):

```

```

985         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
986         ``total_cost_inr`` + the
987         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
988         several."""
989         agg = request.app.state.db.get_video_cost(video_id)
990         fx = request.app.state.config.billing.fx_inr_per_usd
991         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
992         agg["fx_inr_per_usd"] = fx
993         return agg
994
995     @app.get("/api/jobs/{job_id}")
996     def get_job(job_id: str, request: Request):
997         """Job state: {status, progress, result, reports}. `status` = execution state
998         (queued|running|done|failed); the pipeline verdict is result["status"]."""
999         job = request.app.state.db.get_job(job_id)
1000         if not job:
1001             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
1002         result = job.get("result")
1003         return {
1004             "job_id": job["id"],
1005             "status": job["status"],
1006             "progress": job.get("progress"),
1007             "video_id": job.get("video_id"),
1008             "error": job.get("error"),
1009             "result": result,
1010             "reports": (result or {}).get("reports"),
1011             "created_at": job.get("created_at"),
1012             "started_at": job.get("started_at"),
1013             "finished_at": job.get("finished_at"),
1014         }
1015
1016     # --- Chunked upload (B2, PR-2) -----
1017     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
1018     # in
1019     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
1020     # via
1021     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
1022     # sequential-part logic, and abort cleanup ? is unchanged.
1023     from backend.api import uploads as uploads_api
1024
1025     app.include_router(uploads_api.router)
1026
1027     # --- Highlight download (B3, PR-2) -----
1028
1029     @app.get("/api/highlights/{video_id}/{filename}")
1030     def download_highlight(video_id: str, filename: str, request: Request):
1031         """Stream a compiled highlight reel ? `filename` is the bare name given to
1032         /api/compile
1033         (which only writes into output_dir; the browser previously got back an unreachable
1034         server-local path)."""
1035         try:
1036             name = safe_output_filename(filename)
1037         except ValueError as e:
1038             raise HTTPException(status_code=400, detail=str(e)) from e
1039         if not request.app.state.db.get_video(video_id):
1040             raise HTTPException(status_code=404, detail="Indexed video record not found.")
1041         output_dir = (
1042             getattr(
1043                 getattr(request.app.state.config, "output", None), "output_dir", "output"
1044             )
1045             or "output"

```

```

1045     )
1046     path = os.path.join(output_dir, name)
1047     try:
1048         path = resolve_under_root(path, output_dir)
1049     except ValueError as e:
1050         raise HTTPException(status_code=400, detail=str(e)) from e
1051     if not os.path.isfile(path):
1052         raise HTTPException(
1053             status_code=404,
1054             detail=f"No compiled file named '{name}'. Compile first via /api/compile.",
1055         )
1056     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
1057     return FileResponse(path, media_type=media, filename=name)
1058
1059
1060 @app.post("/api/query")
1061 def query_play_segments(req: QueryRequest, request: Request):
1062     filters = {
1063         "server": req.server,
1064         "receiver": req.receiver,
1065         "duration_min": req.duration_min,
1066         "event_type": req.event_type,
1067     }
1068     segments = request.app.state.db.query_play_segments(req.video_id, filters)
1069     return {"play_segments": segments}
1070
1071
1072 @app.post("/api/compile", status_code=202)
1073 def compile_highlights(req: CompileRequest, request: Request):
1074     """Submit a reel-COMPILE job (202 + job_id); poll GET /api/jobs/{job_id} for the
1075     reel (#329).
1076
1077     Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip +
1078     concat + watermark)
1079     can run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing it
1080     synchronously returned a
1081     Cloudflare 524 while the engine kept going (the reel built, but the client saw
1082     "failed" and never
1083     got the URL). The cheap checks (video exists, segments match + survive the floor,
1084     safe filename)
1085     fast-fail HERE with a 4xx; the stitch runs on the worker. On `done`, the job's
1086     `result` carries
1087     {filepath, download_url} ? the same shape the sync endpoint returned, now under
1088     /api/jobs/{id}."""
1089     # M9 edge-auth tenant (DEFAULT-OFF ? None); a missing/spoofed identity fails 401
1090     before queuing.
1091     try:
1092         owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
1093     except edge_auth.EdgeAuthError as e:
1094         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
1095     try:
1096         video, _kept, out_name = _resolve_compile(
1097             request.app.state.db,
1098             request.app.state.config,
1099             req.video_id,
1100             req.segment_ids,
1101             req.output_filename,
1102         )
1103     except _CompileError as e:
1104         raise HTTPException(status_code=e.status_code, detail=e.detail) from e
1105
1106     job_id = uuid.uuid4().hex
1107     # Persist the video's SOURCE ref (the NOT NULL video_path); the worker re-fetches
1108     the row by id so
1109     # the freshest filepath wins. owner_id/locale ride along for M9 multi-tenant + the

```

```

localized reel.
1101         if not request.app.state.db.create_compile_job(
1102             job_id,
1103             req.video_id,
1104             video["filepath"],
1105             req.segment_ids,
1106             out_name,
1107             locale=req.locale,
1108             owner_id=owner_id,
1109         ):
1110             raise HTTPException(
1111                 status_code=500, detail="Failed to persist compile job record."
1112             )
1113         _get_executor().submit(
1114             _drain_due_jobs, request.app.state.config, request.app.state.db
1115         )
1116         return JSONResponse(
1117             status_code=202,
1118             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
1119         )
1120
1121
1122 # --- CLI Debug Utility & Orchestration ---
1123 def run_cli_diagnose_clip(video_path: str, timestamp: float):
1124     """CLI debugging utility to examine segment properties around a timestamp."""
1125     print("=" * 60)
1126     print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
1127     print("=" * 60)
1128
1129     cfg = load_config() # legacy wrapper still works, returns dict for now
1130
1131     # 1. Validation check diagnostics
1132     print("[DIAGNOSTIC] Running validation framework...")
1133     results = registry.run_all(video_path, cfg)
1134     for r in results:
1135         status_symbol = "[PASS]" if r.passed else "[FAIL]"
1136         print(f"  {status_symbol} {r.validator_name}: {r.message}")
1137         if r.details:
1138             print(f"    Details: {r.details}")
1139
1140     # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
1141     print("-" * 60)
1142     print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
1143     # Lazy import: cv2 is NOT installed in the LIGHT tier (torch-free default)
1144     # and this --debug-clip path is CLI-only, not on the served API path.
1145     import cv2
1146
1147     cap = cv2.VideoCapture(video_path)
1148     if not cap.isOpened():
1149         print("[FAIL] OpenCV could not open video.")
1150         return
1151
1152     fps = cap.get(cv2.CAP_PROP_FPS)
1153     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
1154
1155     start_frame = max(0, int((timestamp - 3.0) * fps))
1156     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
1157
1158     # Measure frame-by-frame changes in sample region
1159     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
1160     prev_gray = None
1161     motions = []
1162
1163     for _ in range(start_frame, end_frame):
1164         ret, frame = cap.read()

```

```

1165         if not ret:
1166             break
1167         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
1168         gray = cv2.GaussianBlur(gray, (21, 21), 0)
1169
1170         if prev_gray is not None:
1171             diff = cv2.absdiff(prev_gray, gray)
1172             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
1173             motion_ratio = cv2.countNonZero(thresh) / (
1174                 thresh.shape[0] * thresh.shape[1]
1175             )
1176             motions.append(motion_ratio)
1177             prev_gray = gray
1178
1179     cap.release()
1180
1181     if motions:
1182         avg_motion = sum(motions) / len(motions)
1183         # Support both legacy dict (from wrapper) and future direct model
1184         if hasattr(cfg, "indexing"):
1185             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
1186         else:
1187             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
1188         print(f" Sample Frame Count: {len(motions)}")
1189         print(
1190             f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})"
1191         )
1192         if avg_motion > threshold:
1193             print(
1194                 " Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play."
1195             )
1196         else:
1197             print(
1198                 " Result: [DEAD TIME] Idle movement. Segment classified as dead time."
1199             )
1200     else:
1201         print(" [ERROR] No visual frames were extracted in the sample range.")
1202
1203     print("=" * 60)
1204
1205
1206     def _enforce_startup_or_exit(config) -> None:
1207         """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL
finding BEFORE
1208         any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
storage) OR, for
1209         the exposed server (``include_exposure=True``, M9 hardening), a half-configured
exposure (edge
1210         auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when
no
1211         ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?
1212         the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted so the
1213         fail-fast wiring + the default-OFF no-op are unit-testable."""
1214         try:
1215             enforce_startup_checks(config, logger=logger, include_exposure=True)
1216         except StartupCheckError as e:
1217             logger.error("[STARTUP] refusing to start: %s", e)
1218             sys.exit(1)
1219
1220

```

```

1221 def _maybe_sweep_gemini_orphans(config) -> None:
1222     """01/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous
crashed run.
1223     A no-op unless the configured provider is Gemini AND its API key is present. Never
raises ?
1224     a failed sweep is logged and swallowed. The Gemini client is built with an HTTP
timeout so
1225     a hung connection can't stall startup indefinitely. Extracted for unit tests."""
1226     try:
1227         provider_name = getattr(config, "ai_provider", "")
1228         if provider_name != "gemini":
1229             return
1230         api_key = os.environ.get("GEMINI_API_KEY")
1231         if not api_key:
1232             return
1233         provider = provider_registry.get(
1234             "gemini", api_key=api_key, model_name=config.ai_model
1235         )
1236         deleted = provider.cleanup_orphaned_files()
1237         if deleted:
1238             logger.info(
1239                 "[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted
1240             )

```

7. user (2026-07-04T01:48:42.846Z)

```

1     """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3     Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4     proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
5     under security.upload_dir, then submitted to /api/process by the returned path.
6     Upload state is in-process by design (this is the single-box alpha): a server restart
7     aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8     arrives with PR-3 identity scoping.
9
10    The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11    ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12    reads the live value the server set at startup (and so tests that monkeypatch
13    ``backend.main.global_config`` flow through unchanged).
14    """
15
16    import os
17    import threading
18    import unicodedata
19    import uuid
20    from typing import Any, Dict, Optional
21
22    from fastapi import APIRouter, HTTPException, Request
23    from pydantic import BaseModel
24
25    from backend.utils.app_home import (
26        resolve_output_dir, # P0a: app-home-aware path resolution
27    )
28    from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
29    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
30    from backend.utils.paths import safe_output_filename
31
32    router = APIRouter()
33
34    _uploads: Dict[str, Dict[str, Any]] = {}
35    _uploads_lock = threading.Lock()
36
37
38    class UploadInitRequest(BaseModel):
39        filename: str

```



```

40         total_size_bytes: Optional[int] = None
41
42
43     class UploadCompleteRequest(BaseModel):
44         total_parts: int
45
46
47     def _upload_cfg():
48         # Resolved lazily (import inside the function) to read the LIVE startup config that
49         # main() sets on the module global, and to avoid a circular import at module load
50         # (main.py imports this router; this module must not import main at import time).
51         import backend.main as _main
52
53         sec = getattr(_main.global_config, "security", None)
54         upload_dir = resolve_output_dir(
55             getattr(sec, "upload_dir", None) or "output/uploads"
56         ) # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
57         max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
58         part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
59         exts = [
60             e.lower().lstrip(".")
61             for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
62         ]
63         return upload_dir, max_bytes, part_bytes, exts
64
65
66     def _abort_upload(upload_id: str) -> None:
67         with _uploads_lock:
68             st = _uploads.pop(upload_id, None)
69         if st:
70             try:
71                 os.remove(st["tmp_path"])
72             except OSError:
73                 pass
74
75
76     def _normalize_upload_filename(filename: str) -> str:
77         """Normalize browser-provided file names before extension validation.
78
79         Some mobile/file-provider pickers can hand us visually normal names with
80 compatibility
81 characters (for example a fullwidth dot) or trailing whitespace/dots. Keep the
82 existing
83 traversal/null-byte checks in ``safe_output_filename`` as the authority after this
84 cleanup.
85
86 """
87         return unicodedata.normalize("NFKC", filename).strip().rstrip(". ")
88
89
90     @router.post("/api/upload/init")
91     def upload_init(req: UploadInitRequest):
92         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
93         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
94         try:
95             name = safe_output_filename(_normalize_upload_filename(req.filename))
96         except ValueError as e:
97             raise HTTPException(status_code=400, detail=str(e)) from e
98         ext = os.path.splitext(name)[1].lower().lstrip(".")
99         if ext not in exts:
100             raise HTTPException(
101                 status_code=400,
102                 detail=f"Extension '{ext}' not allowed. Allowed: {sorted(exts)}",
103             )
104         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
105             raise HTTPException(

```

```

102         status_code=413,
103         detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
104     )
105     os.makedirs(upload_dir, exist_ok=True)
106     # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
107     effort ? an
108     # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
109     (never blocks a
110     # legit upload); the per-part total cap in upload_part is the hard backstop either
111     way.
112     require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
113     upload_id = uuid.uuid4().hex
114     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
115     open(tmp_path, "wb").close()
116     with _uploads_lock:
117         _uploads[upload_id] = {
118             "filename": name,
119             "tmp_path": tmp_path,
120             "parts": 0,
121             "bytes": 0,
122         }
123     return {
124         "upload_id": upload_id,
125         "max_part_mb": part_bytes // (1024 * 1024),
126         "next_part": f"/api/upload/{upload_id}/part/0",
127     }
128
129 @router.put("/api/upload/{upload_id}/part/{index}")
130 async def upload_part(upload_id: str, index: int, request: Request):
131     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
132     _, max_bytes, part_bytes, _ = _upload_cfg()
133     with _uploads_lock:
134         st = _uploads.get(upload_id)
135         if st is None:
136             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
137         if index != st["parts"]:
138             raise HTTPException(
139                 status_code=409,
140                 detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
141                 sequential)",
142             )
143         received = 0
144         overflow = None
145         with open(st["tmp_path"], "ab") as fh:
146             async for chunk in request.stream():
147                 received += len(chunk)
148                 if received > part_bytes:
149                     overflow = (
150                         f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
151                     )
152                     break
153                 if st["bytes"] + received > max_bytes:
154                     overflow = (
155                         f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
156                     )
157                     break
158                 fh.write(chunk)
159         if overflow:
160             _abort_upload(upload_id)
161             raise HTTPException(
162                 status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
163             )

```

```

163         with _uploads_lock:
164             st["parts"] += 1
165             st["bytes"] += received
166         return {
167             "upload_id": upload_id,
168             "received_parts": st["parts"],
169             "received_bytes": st["bytes"],
170         }
171
172
173     @router.post("/api/upload/{upload_id}/complete")
174     def upload_complete(upload_id: str, req: UploadCompleteRequest):
175         """Finalize: verify part count, move into place, return {path, video_id} for
176         /api/process."""
177         upload_dir, _, _, _ = _upload_cfg()
178         with _uploads_lock:
179             st = _uploads.get(upload_id)
180             if st is None:
181                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
182             if st["parts"] == 0 or st["parts"] != req.total_parts:
183                 raise HTTPException(
184                     status_code=409,
185                     detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
186                 )
187             final_path = os.path.join(upload_dir, st["filename"])
188             if os.path.exists(
189                 final_path
190             ): # never overwrite an earlier upload with the same name
191                 stem, ext = os.path.splitext(st["filename"])
192                 final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
193             os.replace(st["tmp_path"], final_path)
194             with _uploads_lock:
195                 _uploads.pop(upload_id, None)
196             video_id = compute_video_id(final_path)
197             return {
198                 "path": os.path.abspath(final_path),
199                 "video_id": video_id,
200                 "size_bytes": os.path.getsize(final_path),
201                 "next": "POST /api/process with this path",
202             }
203
204     @router.delete("/api/upload/{upload_id}")
205     def upload_abort(upload_id: str):
206         """Abort an in-flight upload and delete its partial file."""
207         with _uploads_lock:
208             known = upload_id in _uploads
209             if not known:
210                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
211             _abort_upload(upload_id)
212             return {"status": "aborted", "upload_id": upload_id}
213

```

8. assistant (2026-07-04T01:48:46.714Z)

I have all the information I need. The routes are fully defined in these two files. Let me note the response body for `/api/process` errors and confirm there's no separate `/api/download/{name}` (downloads are `/api/highlights/{video\_id}/{filename}` and `/api/reels/{filename}`). I have everything needed.

9. user (2026-07-04T01:49:23.124Z)

Structured output provided successfully